

1 Questions

- A massive problem in deep learning now is that we don't have any clue how these neural networks actually work once trained.
- Are there efficient hyperparameter optimization algorithms?
- What are graph neural networks?
- Are there CNN architectures that don't require fixed input sizes?

2 Interesting

- Distributed model training
- Reducing the memory requirements of a model during training
- Double descent

3 Math concepts

$Pr(x)$, a probability density function, assigns a non-negative probability density to every value in the input domain. The normalization condition is $\int Pr(x) dx = 1$. We assume random variables are continuous.

A joint probability distribution, $Pr(x, y)$ (probability of x and y) assigns a probability to every combination of values and $\int \int Pr(x, y) dx dy = 1$.

A marginal distribution is the probability of a variable while completely ignoring other variables.

$$Pr(\mathbf{X} = x) = \sum_y Pr(X = x, Y = y)$$

A conditional probability $Pr(x|y)$ is the probability of x given y .

$$Pr(x|y) = \frac{Pr(x, y)}{Pr(y)}$$

KL Divergence is a measure of how much the test distribution diverges from the correct distribution. Remember that entropy is just a weighted sum of surprisal. We subtract cross entropy from entropy. Ideally, the two distributions $p(x)$ (actual) and $q(x)$ (test) are equal, so the entropy and the cross entropy are equal, and the divergence is zero.

$$D_{KL}[P||Q] = \int_{-\infty}^{\infty} p(x) \log \frac{P(x)}{Q(x)}$$

A Jacobian matrix is a matrix storing the derivative of one vector with respect to another. For example:

$$f(x, y) = \begin{bmatrix} x + y \\ xy \end{bmatrix}$$
$$J = \frac{\partial f}{\partial (x, y)} = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ y & x \end{bmatrix}$$

The sigmoid function maps a real value z to a range of $[0, 1]$.

$$\text{sig}[z] = \frac{1}{1 + \exp[-z]}$$

The softmax function maps a vector of real values (logits) $\mathbf{z}$ to a vector of probabilities in a range of 0 to 1, where all probabilities sum to 1.

$$\text{softmax}[\mathbf{z}_k] = \frac{\exp[z_k]}{\sum_{k'=0}^K \exp[z_{k'}]}$$

Suppose a transformation $t[x]$ is applied on a function $f[x]$. The function is **invariant** if $f[t[x]] = f[x]$ and is **equivariant** if $f[t[x]] = t[f[x]]$.

4 Neural network basics

A model is a mathematical function defined by parameters that maps an input to an output. There are discriminative models and generative models. Discriminative models find outputs from inputs, while generative models find inputs from outputs. Most models are discriminative since it's easier to exploit the prior knowledge that's baked into the input data. Generative models have much higher losses than discriminative models, and seem to perform worse on the demo examples. Trying to capture all of the nuance that goes into generating a specific output really is so much harder than mapping an input to an output.

Using an activation function allows a model to represent non linear functions. The activation function has to be non linear in order to be effective. A common activation function is ReLU (Rectified Linear Unit). If the value is ≤ 0 , the output is 0, else it's the value, which clamps negative numbers.

Network capacity is the model's ability approximate different functions. The network capacity is defined as the number of parameters, the number of layers (depth), the number of hidden units in each layer, etc. The theoretical capacity is called the representational capacity, while the actual number of functions that the model can approximate is called the effective capacity. With enough capacity (hidden units), the network can approximate any function at a given resolution (Universal approximation theorem).

Parameters are layer weights and biases, while hyperparameters are high level settings on how the neural network operates (ex: learning rate, epoch, batch size, number of layers, hidden units per layer, etc). A common way to optimize hyperparameters is to split the training dataset into three subsets. A training set, a validation set, which is used to choose the hyperparameters that lead to the best model performance, and a test set which is used for a final model performance evaluation. Another common approach is called k-fold cross validation. Training samples are split into N random subsets, and for N iterations, training is done with $N - 1$ subsets while hyperparameter validation is done with the remaining subset. The final model performance is evaluated using an average of the N model predictions on a different test dataset.

There are three main sources of error when training a model: The extent to which model performance is maintained on the training set is called generalization.

- **Noise:** The inherent randomness (mislabelling, multiple inputs mapping to the same output, etc) in the training dataset is insurmountable.
- **Bias:** The model might not have enough capacity to fit the data accurately. Increasing the model capacity decreases bias but increases variance, and doesn't necessarily decrease test error. This is known as the bias-variance tradeoff. Increasing model capacity unfortunately makes it model noise in the training data better, which is referred to as overfitting.
- **Variance:** The fitted function will vary slightly based off of the training samples (and possibly SGD). Increasing the number of training samples decreases variance.

As model capacity increases, test error should decrease up until the point where variance becomes too large and the model starts to overfit, making the test error increase. However, for many datasets, the test error starts decreasing again as the model capacity continues to increase. This is referred to as double descent. At the moment we don't fully know why, but there's at least one plausible explanation. As the model capacity increases and the training error approaches zero, the model's fitting function is able to pass through every single training sample. The segments between data points are initially very erratic, leading to a large test error. However, as the model capacity continues to grow, the segments between data points become smoother and smoother, which makes the model fit to new test points better, decreasing test error. This is especially plausible when you consider that in a high dimensional input space, the training samples are extremely sparse

(curse of high dimensionality). There's a large number of possible fitting functions passing through all training samples that the model can adopt. The model's tendency to adopt certain families of fitting functions is called inductive bias. Any factor that biases a solution to a set of equivalent solutions is called a regularizer. The hypothesis is that SGD is an implicit regularizer of smooth functions.

Expectation is a weighted sum of all the possible values a random value could be. If the expectation of the magnitude of the weights are smaller than 1, the gradient vanishes, and if the expectation of the magnitude of the weights are larger than 1, the gradient explodes. Both problems can occur in the forward or backward pass. A solution to this is He initialization (also known as Kaiming initialization). Initializing the weights to random values taken from a normal distribution centered around 0, with a variance $\frac{1}{D_h}$, where D_h is the dimension of the current layer, and $D_{h'}$ is the dimension of the next layer, keeps the gradients stable. This particular initialization is used with the ReLU activation function. If the current and next layer have the same dimension, then an average of the variance for the forward pass and backward pass doesn't have to be used (2).

$$\sigma^2 = \frac{4}{D_h + D_{h'}} \quad (1)$$

$$\sigma^2 = \frac{2}{D_h} \quad (2)$$

5 Loss functions

A model outputs a set of parameters (ϕ) that describe a probability distribution. Minimizing the loss function $L[\phi]$ means maximizing $Pr(y|\phi)$. During inference, the final output value is the maximum of the output probability distribution.

$$\begin{aligned} \hat{\phi} &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I Pr(y_i|x_i) \right] \\ &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I Pr(y_i|\theta_i) \right] \\ &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I Pr(y_i|f[\mathbf{x}_i, \phi]) \right] \end{aligned}$$

It's more practical to take the sum of the log of all the probabilities, since they may be very small. Also, by convention the loss function should minimize loss, which can be achieved by multiplying the sum by -1 and using argmin. This is the negative log-likelihood!

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[- \sum_{i=1}^I \log[Pr(y_i|f[\mathbf{x}_i, \phi])] \right]$$

The following are a few common loss functions.

- **Mean squared error** is used in regression tasks, when the output follows a Gaussian distribution. When performing inference, the model's raw output can be used since it is the mean of the output gaussian distribution, which so happens to be the distribution's maximum. Normalization is done to make the loss independent of the number of training samples, and to make comparing loss across datasets easier. I is the number of training samples.

$$L[\phi] = \frac{1}{I} \sum_{i=1}^I (\mathbf{y}_i - f[\mathbf{x}_i, \phi])^2$$

- **Cross entropy loss** is used in classification tasks, when the output follows a categorical distribution. A categorical distribution is a distribution where the probability of each class is $\lambda_k \in [0, 1]$ and $\sum_K \lambda_k = 1$. To ensure that the model output follows a categorical distribution, softmax is applied. y is the true class, $\hat{y}_i$ is the predicted class.

$$L[\phi] = - \sum_{i=1}^I \sum_{k=1}^K y_i \log(\hat{y}_i)$$

- When the model's output is multivariate, each output and each error are treated as independent. The model's loss function becomes a sum of all the loss functions for each output. See the chart on p. 70 of textbook for more.

6 Backpropagation

Backpropagation is an algorithm that iteratively applies the chain rule to any arbitrary computational graph. A neural network can be thought of as a series of function compositions, where each layer is a function that composes the previous layer. A neural network can also be thought of as a directed acyclic graph, where each layer is a node. Backpropagation is just going through each node, right to left, and computing the local gradient, multiplying it by the global gradient (which is initialized to 1), and passing an updated global gradient to child nodes (previous layers). Gradients are computed for every training sample in a batch, then summed together to get the gradient used in an optimization algorithm. The loss function gradient is used as the initial gradient.

A forward pass is done to compute and cache values of hidden units in each layer, which will be used to compute gradients. Each sample in the batch uses the same weights and the same biases. Layers are different for every sample in the batch, and weight and bias gradients are summed across the batch (don't have a batch dimension).

$$\begin{aligned} f_0 &= \beta_0 + \Omega_0 \mathbf{x} \\ h_k &= a[f_{k-1}] & k \in \{1, 2, 3, \dots, K\} \\ f_{k-1} &= \beta_k + \Omega_k \mathbf{h}_k & k \in \{1, 2, 3, \dots, K\} \end{aligned}$$

A backward pass is done to compute the derivative of the loss with respect to every weight and bias in the network. The rules for computing local gradients and updating the global gradient are as follows. Note that an outer product is used when computing the derivative of loss with respect to weights. The derivative of the loss with respect to the hidden units is a column vector. Gradients are layer specific, and during the optimization algorithm, the gradient used to step is an average of all the gradients for each training sample in the batch.

$$\begin{aligned} \frac{\partial l_i}{\partial \beta_k} &= \frac{\partial l_i}{\partial f_k} \\ \frac{\partial l_i}{\partial \Omega_k} &= \frac{\partial l_i}{\partial f_k} h_k^T \\ \frac{\partial l_i}{\partial f_{k-1}} &= I[f_{k-1} > 0] \odot \Omega_k^T \frac{\partial l_i}{\partial f_k} \end{aligned}$$

$k \in \{K-1, K-2, K-3, \dots, 1\}$, and $\frac{\partial l_i}{\partial f_k}$ is the global gradient. The derivative of ReLU with respect to the hidden units is a Jacobian matrix (J). When $i \neq j$, $J_{ij} = 0$, since $\text{ReLU}'(i)$ only depends on i . The diagonal elements in J would be either 1 or 0 (considering the definition of ReLU). Instead of computing the entire Jacobian, its diagonal elements are directly computed, which is just: `hidden_unit > 0`.

Backpropagation is different for convolutional layers. We need to compute the derivative of the loss with respect to the kernel F (1) and the derivative of the loss with respect to the previous layer X (2). (1) X is padded by the current layer's padding and the gradient is dilated by the current layer's stride. (2) To rotate by 180 degrees, F is flipped vertically then horizontally. The gradient dilated by the current layer's stride and zero-padded by the current layer's kernel size minus one. After the convolution, the gradient should have its padding stripped.

$$\frac{\partial L}{\partial F} = \text{convolution} \left[X, \frac{\partial l_i}{\partial f_k} \right] \quad (1)$$

$$\frac{\partial L}{\partial X} = \text{convolution} \left[\frac{\partial l_i}{\partial f_k}, \text{rot180}[F] \right] \quad (2)$$

$$\frac{\partial L}{\partial B_k} = \sum_{y=1}^H \sum_{x=1}^W \frac{\partial l_i}{\partial f_{k,y,x}} \quad (3)$$

7 Optimization algorithms

The goal of an optimization algorithm is to update the model's parameters as to minimize the loss. This process is known as learning, training or fitting.

- **Gradient descent:** Gradients are the partial derivatives of the loss function with respect to the model parameters. α is the learning rate, which is a hyperparameter that controls how quickly the parameters change. A learning rate scheduler like StepLR is often used (γ is a decay multiplier). The problem with gradient descent is that there's no way of knowing whether the model will converge on a global minimum or a local minimum, or get stuck in a saddle point (flat area of the function) especially when the model function is non-convex.

$$\alpha_t = \alpha_0 * \gamma^{\frac{\text{epochs}}{\text{step_size}}}$$

$$\hat{\phi} = \phi - \alpha \frac{\partial L}{\partial \phi}$$

- **Stochastic gradient descent:** SGD adds randomness to the stepping process by sampling a random subset of training samples (batch) every epoch. Because of the batching, SGD is more efficient. Since the loss function for each batch is different, gradients vary, which makes the model take a noisier path to convergence. This allows the parameters to potentially step out of a local minima or a saddle point.

$$\hat{\phi} = \phi - \alpha \sum_{i \in B_t} \frac{\partial l_i[\phi_i]}{\partial \phi}$$

Where B_t is the current batch of training samples and l_i is the loss given the chosen training samples. SGD generalizes better than gradient descent, and smaller batch sizes result in better performance. This is because SGD implicitly penalizes variance, making the model learn a function that's a tighter, smoother fit to the data.

- **Nesterov accelerated momentum:** By adding a weighted combination of the previous gradients, and the gradients at the target position, the model can converge on a minimum much faster. Notice that the momentum terms are recursive, and older momenta get weighed less and less. Past gradients are weighed more than the current gradients. By computing the gradient at the target position we also avoid the overshooting that would happen if we used the gradients at the starting position.

$$m_{t+1} = B * m_t + (1 - B) \sum_{i \in B_t} \frac{\partial l_i[\phi_t - \alpha B * m_t]}{\partial \phi}$$

$$\hat{\phi} = \phi_t - \alpha * m_{t+1}$$

- **Adam:** The Adam optimizer takes ideas from several other algorithms and combines them into one. First, Include a weighted history of both the momentum and the squared momentum. Second, amplify the momentum and squared momentum when t is small, doing nothing when t is large. This is because the momentum (which is initialized to 0) will be too small when t is small. Lastly, normalize the gradients so that the parameters step the same distance in each direction. Momentum is preserved across batches

and epochs.

$$m_{t+1} = \beta m_t + (1 - \beta) \sum_{i \in B_t} \frac{\partial l_i(\phi_t)}{\partial \phi}$$

$$v_{t+1} = \gamma v_t + (1 - \gamma) \left(\sum_{i \in B_t} \frac{\partial l_i(\phi_t)}{\partial \phi} \right)^2 \quad (1)$$

$$\tilde{m}_{t+1} = \frac{m_{t+1}}{1 - \beta^{t+1}}$$

$$\tilde{v}_{t+1} = \frac{v_{t+1}}{1 - \gamma^{t+1}} \quad (2)$$

$$\hat{\phi} = \phi_t - \alpha \frac{\tilde{m}_{t+1}}{\sqrt{\tilde{v}_{t+1}} + \epsilon} \quad (3)$$

8 Regularization

See p.155, figure 9.14

Regularization refers to any method that decreases the generalization gap between test and training performance. Explicit regularization involves adding an explicit term to the loss function in order to favor certain parameters. This involves moving away from a maximum likelihood criterion to a maximum a posteriori criterion, where the negative log loss also includes a probability $Pr(\phi)$, a belief about the data.

$$\hat{\phi} = \operatorname{argmax}_{\phi} \left[Pr(\phi) \prod_{i=1}^I Pr(y_i | x_i, \phi) \right]$$

The most common regularization term is the **L2 norm**, which biases smaller parameters, making the model output change slower. Parameters should be as small as possible (simplest solution) while still minimizing the loss function. L2 norm is applied to weights, and having smaller weights will reduce variance and make the model fit a smoother function, at the cost of reducing data fit. With small values of λ , test performance improves.

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[\sum_{i=1}^I l_i[x_i, y_i] + \lambda \sum_j \phi_j^2 \right]$$

The following are ways to improve model performance. Note that these aren't silver bullets. In theory they should improve test performance, but neural networks are very finicky; the optimal hyperparameters and learning strategies to use should be determined empirically.

- **Early stopping:** Stop training the model before it fully converges. In practice this means training the model for a large number of epochs to see which number of epochs maximizes performance.
- **Ensembling:** Run inference on multiple models and average their outputs together to get a more confident answer. This can be done by training identical models with random parameter initializations, identical models trained on different randomly sampled training data (called bootstrap aggregating), or training different model architectures on identical data.
- **Dropout:** During training, clamp a random subset of hidden units (typically half) to zero. This helps the model reduce its reliance on any given hidden unit and encourages the weights to be smaller so that the output fluctuates less wildly when hidden units are turned off. During inference, all hidden units are active, so weights are multiplied by $1 - \eta$, where η is the dropout probability (weight scaling inference rule). One can also do Monte Carlo dropout instead, where you run inference on the model multiple times with random subsets of the hidden units clipped, and then combine results.
- **Apply noise to:**
 - Inputs: Makes the model fit to more variable samples, improving test performance.

- **Weights:** Makes the training converge to a local minima inside of a flat middle region, where changing the weights doesn't affect the output much. The model becomes less particular about specific weight values.
- **Labels:** Makes the model stop chasing absolute certainty. Label smoothing refers to changing the loss function to minimize the cross-entropy between the predicted and actual distribution (Where the true label has a probability $1 - \rho$ and all the other classes have equal probability. ρ is the proportion of the training labels that are assumed incorrect.).
- **Transfer learning:** The initial layers of a model act as feature extractors and the final layers act as classification heads. Transfer learning leverages this by freezing the parameters of the pre-trained feature extractor and only training the classification head on the target dataset. Multi-task learning is when the model is trained to solve multiple problems concurrently, hoping that the performance on each will improve.
- **Data Augmentation:** Generate different variations of the input data (add synonyms to text, blur images, filter audio, etc) in order to teach the model to be indifferent to irrelevant data transformations.
- **Self-supervised learning:** Generate more labelled data using a secondary generative model.
 - In generative self-supervised learning, parts of each data is masked and the secondary task is to predict the missing part.
 - In contrastive self-supervised learning, pairs of examples with commonalities are compared to unrelated pairs.

9 Convolutional networks

A convolutional network is composed of a series of convolutional layers. Convolutional layers are a special subset of fully connected layers. A convolution is a weighted sum over a set of inputs using fixed parameters called a convolutional kernel or a filter. A valid convolution is a convolution that avoids inputs that are out of range, and a zero padded convolution is a convolution that uses zeros in place of inputs that are out of range. Stride is the step size used when sliding over the input and dilating is the number of zeros interspersed between the inputs plus one.

A convolutional layer computes its outputs by convolving the input, adding a bias and passing the result through an activation function. Each kernel used to convolve corresponds to an output channel. Channels add depth to the convolutional layer. A convolutional layer is equivariant to translation because of the use of the same kernel over the entire image, and partially invariant to translation because of max pooling.

For a 2D convolution on RGB images, a $C_i \times K \times K$ kernel is slid over the image, resulting in a $C_h \times C_w$ matrix. C_i is the number of input channels, C_w is the output width, C_h is the output height. Hidden layers are 3D, unlike layers in feedforward networks that are always 1D. Each channel's hidden unit is computed as the following, where ω_{ij} is the kernel weights, m is $\lceil K/2 \rceil$ and $n \in [1, C_o]$.

$$h_{nyx} = a \left[\beta + \sum_{c=1}^{C_i} \sum_{i=1}^K \sum_{j=1}^K \omega_{cij} x_{c,i-m,j-m} \right]$$

Weights are defined as $\Omega \in \mathbb{R}^{C_o \times C_i \times K \times K}$ where C_i is the number of channels in the previous layer, C_o is the number of channels in the current layer and K is the kernel size. Biases can be defined as $\beta \in \mathbb{R}^{C_o}$. The output layer size is defined as the following where N is the current layer size, P is padding and K is the kernel size and S is the stride.

$$\left\lfloor \frac{N + 2P - K}{S} + 1 \right\rfloor$$

There are different types of convolutions:

- **Strided convolution:** Refers to a convolution with a stride > 1 . This downsamples the original input size. For example, with a stride of 2, there would be half as many outputs as there are inputs. There are other ways to downsample an image. Using max pooling/mean pooling reduces a 2x2 block of pixels down to the max value/mean value of the block.

- **Transposed convolution:** Refers to a convolution that uses the transpose of the weight matrix used in a strided convolution. This upsamples the original input size. For example, there would be twice as many outputs as there are inputs, and each input would correspond to two outputs. There are other ways to upsample an image. Using max unpooling or bilinear interpolation or duplicating the current value across a 2x2 block.
- A **1x1 convolution** reduces the number of feature maps without reducing the number of spatial dimensions. A $C_o \times C_i \times 1 \times 1$ kernel is ran on the input layer, resulting in a $1 \times C_h \times C_w$ result for every output channel C_o . Since this mixes channels this also adds non linearity.

10 Residual Networks

Deeper networks exhibit a degradation problem: the more layers are added, the higher the training/test error because the model's fitted function is more variable.

A group of consecutive layers is referred to as a block. In the paper, the block is composed of 3 layers: 1x1 convolutions, 3x3 convolutions and 1x1 convolutions. Instead of hoping that the block will fit an underlying mapping $H(x)$, we make the block directly fit a residual mapping $F(x) = H(x) - x$, where the underlying mapping becomes $H(x) = F(x) + x$. $F(x)$ is a layer output (linear combination, convolution, etc) and $+x$ refers to adding the block input to the block output.

The dimensions of $F(x)$ and x must be the same. If not, a linear projection of x is used. The element wise addition of an input and output convolutional layer is done channel by channel. If the channel counts aren't the same, zero padding is used (adding no extra parameters) or 1x1 convolutions are used (adding more parameters). 1x1 convolutions produce better results. If the dimensions aren't the same, the element wise addition is done in a strided way to downsample the input.